

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №2

«Рефакторинг и тесты вариант 21 А»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Подготовить решение РК-1 к модульному тестированию и реализовать три теста с использованием TDD-фреймворка. Предметная область — Оператор и Язык программирования, вариант 21, запросы А. В работе используется стандартный модуль unittest.

Выполненный рефакторинг

В РК-1 коллекции, соединения и печать находятся в main. В РК-2 вычисления выделены в query_a1, query_a2 и query_a3. Каждая функция принимает коллекции параметрами, возвращает результат и не читает ввод, не печатает и не меняет исходные данные. Функция main отвечает только за демонстрацию.

Модели представлены dataclass(frozen=True): поля записи нельзя переназначить после создания. sample_data возвращает новые коллекции при каждом вызове. Поэтому изменения тестовых списков в одной проверке не влияют на другую. Названия функций и структура данных сохраняют связь с тремя запросами РК-1.

Три модульных теста

test_a1_join_and_sort проверяет точный результат соединения, сортировку на перевёрнутом входе и пустые коллекции. test_a2_sum_sort_and_empty_groups проверяет суммы, порядок, равные суммы и исключение языка без операторов. test_a3_filter_and_many_to_many проверяет M:N, слово целиком, регистр, пустой список связанных операторов и пустые входные данные.

Ожидаемые значения указаны явно, без вычисления через проверяемые функции. В тесте А3 оператор + должен попасть в Rust через промежуточную таблицу, несмотря на основной язык Python. Это позволяет обнаружить подмену M:N отношением 1:N.

Порядок запуска

```
python3 -m rk_02.solution
python3 -m unittest rk_02.test_queries -v
```

Python 3.10+, запуск из корня pcpl-labs. Установка пакетов не нужна.

Результаты тестирования

```
$ python3 -m unittest rk_02.test_queries -v
test_a1_join_and_sort (rk_02.test_queries.QueryTests.test_a1_join_and_sort) ... ok
test_a2_sum_sort_and_empty_groups
(rk_02.test_queries.QueryTests.test_a2_sum_sort_and_empty_groups) ... ok
test_a3_filter_and_many_to_many
(rk_02.test_queries.QueryTests.test_a3_filter_and_many_to_many) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

Рисунок 1. Экранная форма фактического вывода программы

Изображение построено по сохранённому stdout и stderr. Полные журналы приведены в папке results.

Результат и вывод

Выполнены ровно три тестовых метода, все завершились успешно. Внутри методов проверено несколько сценариев. Рефакторинг сохранил результаты РК-1; вывод программ сохранён в results/demo.txt. Разделение вычислений и вывода позволяет проверять запросы на разных данных без запуска консольного сценария.

Источник задания

Условия РК-2. Проверено 19 октября 2026 года.

https://github.com/ugapanyuk/courses_current/blob/main/python/rk/pcpl/pcpl_rk_2.pdf

Текст программы

```
# Файл solution.py

"""Рефакторинг РК-1: неизменяемые данные и чистые функции запросов."""

from dataclasses import dataclass
from operator import itemgetter


@dataclass(frozen=True)
class Operator:
    id: int
    name: str
    arity: int
    language_id: int


@dataclass(frozen=True)
class Language:
    id: int
    name: str


@dataclass(frozen=True)
class OperatorLanguage:
    operator_id: int
    language_id: int


def sample_data():
    """Каждый вызов возвращает отдельные коллекции тестовых данных."""
    return (
        [Operator(1, "+", 2, 1), Operator(2, "*", 2, 1),
         Operator(3, "not", 1, 3), Operator(4, "-", 2, 2)],
        [Language(1, "язык Python"), Language(2, "язык Rust"),
         Language(3, "язык Ada")],
        [OperatorLanguage(1, 1), OperatorLanguage(1, 2),
         OperatorLanguage(2, 1), OperatorLanguage(3, 3),
         OperatorLanguage(4, 2)],
    )


def query_a1(operators, languages):
    """Соединение 1:N; сортировка по языку, затем оператору."""
    return sorted(
        [(op.name, op.arity, lang.name)
         for op in operators for lang in languages
         if op.language_id == lang.id],
        key=itemgetter(2, 0),
    )
```

Текст программы Продолжение

```
def query_a2(operators, languages):
    """Сумма арностей; пустые группы исключены, как в примере условия."""
    totals = [
        (lang.name, sum(op.arity for op in operators
                        if op.language_id == lang.id))
        for lang in languages
        if any(op.language_id == lang.id for op in operators)
    ]
    return sorted(totals, key=lambda row: (-row[1], row[0]))


def query_a3(operators, languages, links):
    """Связь M:N независима от language_id основного языка."""
    return {
        lang.name: [op.name for link in links for op in operators
                    if link.language_id == lang.id
                    and op.id == link.operator_id]
        for lang in languages
        if "язык" in lang.name.casefold().split()
    }


def main():
    operators, languages, links = sample_data()
    for label, result in [
        ("A1", query_a1(operators, languages)),
        ("A2", query_a2(operators, languages)),
        ("A3", query_a3(operators, languages, links)),
    ]:
        print(label)
        if isinstance(result, dict):
            for language, names in result.items():
                print(f"{language}: {', '.join(names)}")
        else:
            for row in result:
                print(row)
        print()


if __name__ == "__main__":
    main()

# Файл test_queries.py

"""Ровно три модульных теста на стандартном фреймворке unittest."""

import unittest

from rk_02.solution import (
    Language, Operator, OperatorLanguage,
```

Текст программы Продолжение

```
    query_a1, query_a2, query_a3, sample_data,
)


class QueryTests(unittest.TestCase):
    def test_a1_join_and_sort(self):
        operators, languages, _ = sample_data()
        # Вход намеренно перевёрнут: проверяем ключ сортировки.
        self.assertEqual(query_a1(operators[::-1], languages[::-1]), [
            ("not", 1, "язык Ada"),
            ("*", 2, "язык Python"), ("+", 2, "язык Python"),
            ("-", 2, "язык Rust"),
        ])
        self.assertEqual(query_a1([], languages), [])
        self.assertEqual(query_a1(operators, []), [])

    def test_a2_sum_sort_and_empty_groups(self):
        operators, languages, _ = sample_data()
        languages.append(Language(4, "язык C"))
        self.assertEqual(query_a2(operators, languages), [
            ("язык Python", 4), ("язык Rust", 2), ("язык Ada", 1),
        ])
        # Равные суммы упорядочиваются по названию; пустая группа исключена.
        self.assertEqual(query_a2([
            Operator(1, "+", 2, 1), Operator(2, "-", 2, 2),
        ], languages), [("язык Python", 2), ("язык Rust", 2)])
        self.assertEqual(query_a2([], languages), [])

    def test_a3_filter_and_many_to_many(self):
        operators, languages, links = sample_data()
        self.assertEqual(query_a3(operators, languages, links), {
            "язык Python": ["+", "*"], "язык Rust": ["+", "-"],
            "язык Ada": ["not"],
        })
        # '+' связан с Rust через таблицу M:N, хотя его основной язык Python.
        # Проверяем регистр, слово целиком и язык без связанных операторов.
        languages += [Language(4, "ЯЗЫК C"), Language(5, "метаязык")]
        links += [OperatorLanguage(1, 5)]
        result = query_a3(operators, languages, links)
        self.assertEqual(result["ЯЗЫК C"], [])
        self.assertNotIn("метаязык", result)
        self.assertEqual(query_a3([], languages, []), {
            lang.name: [] for lang in languages
            if lang.id != 5
        })


if __name__ == "__main__":
    unittest.main()
```